

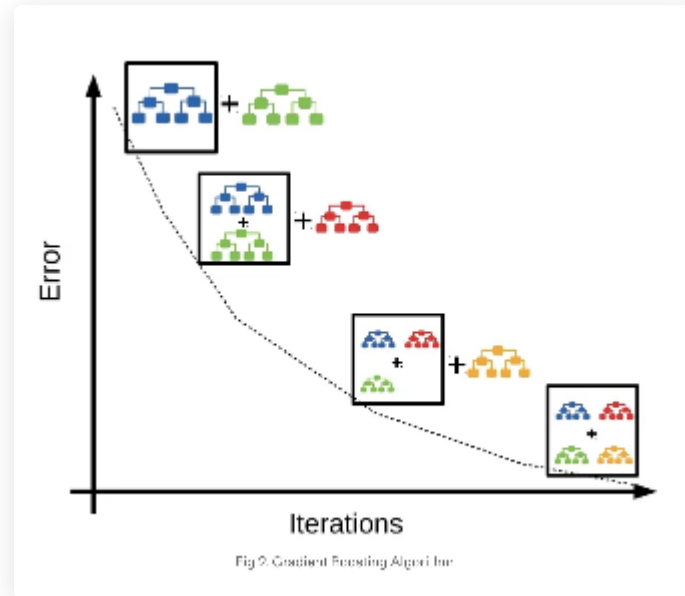
What comes after Deep Learning?

Personal History of Deep Learning

2004 Conditional Random Fields

Training conditional random fields via gradient tree boosting, Tom Dietterich

<https://dl.acm.org/doi/abs/10.1145/1015330.1015428>



2005 Cycle-corrected Belief Propagation [pdf](#)

2013 Google, OCR

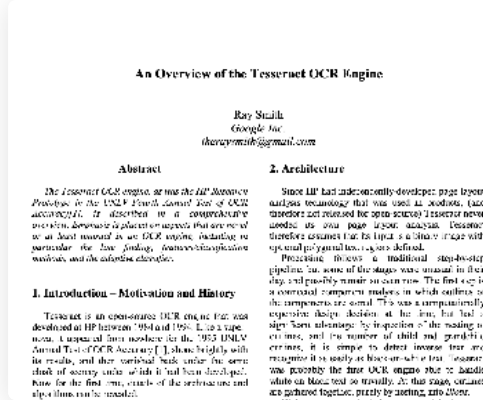
OCR pipeline at Google — rigid hand-built system split across multiple stages, each tuned by different experts. Used as the "before" picture of a bounded, partitioned design.

- Binarization
- Segmentation
- Classification

People really leaned into their respective objectives:
suboptimization

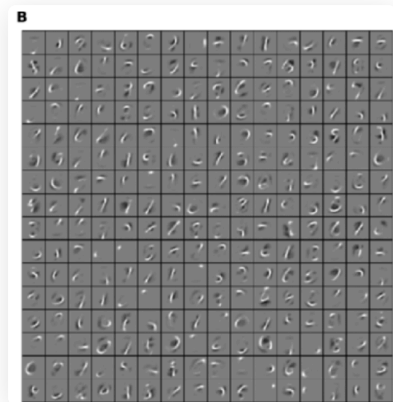


Tesseract OCR engine



Combination of brittle pipelines. Worked well for books after 20 years of tweaking. Failed for outside text.

Andrew Ng comes to Google to teach UFDLD class

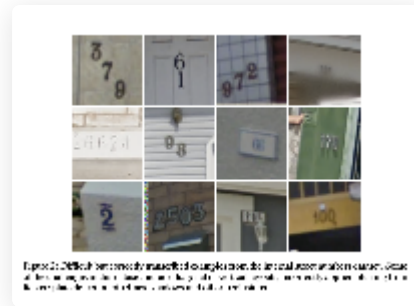


- 1st time implementing backprop by hand

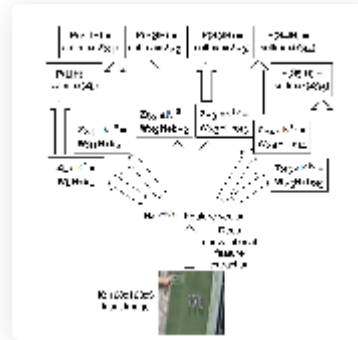
StreetView House Numbers

First DL deployment at Google

Initial reaction skeptical
(we tried deep learning in the 80s, try SIFT)



<https://arxiv.org/abs/1312.6082>



(hired Ian Goodfellow as an intern through this blogpost)

<https://yaroslavvb.blogspot.com/2013/12/deep-learning-internship-at-google.html>

Lesson: removing walls of abstraction opens up new directions

2015 Google Brain Team

DistBelief -> Tensorflow

tensorflow::ops::DiagPart

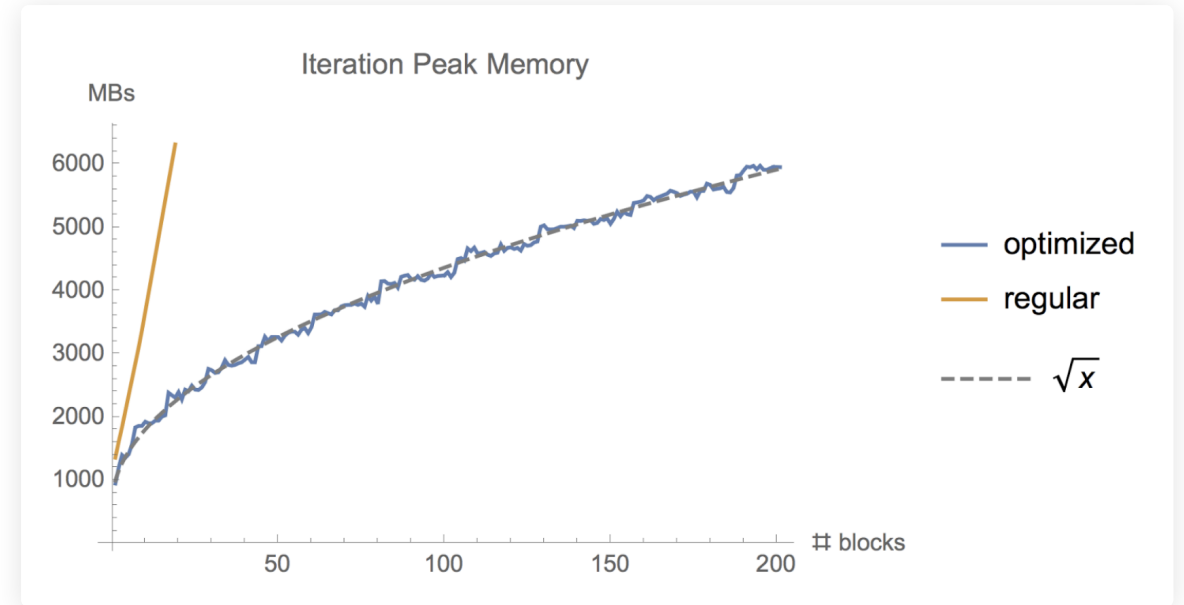
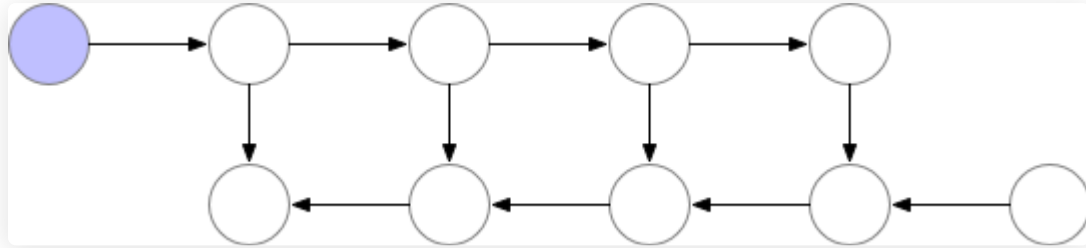
How backprop was implemented originally

- Lua Port
- Rajat
- 2nd-5th time implementing backprop by hand

2018 OpenAI, gradient checkpointing

Gradient checkpointing with Tim Salimans

[cybertronai/gradient-checkpointing](https://github.com/cybertronai/gradient-checkpointing)



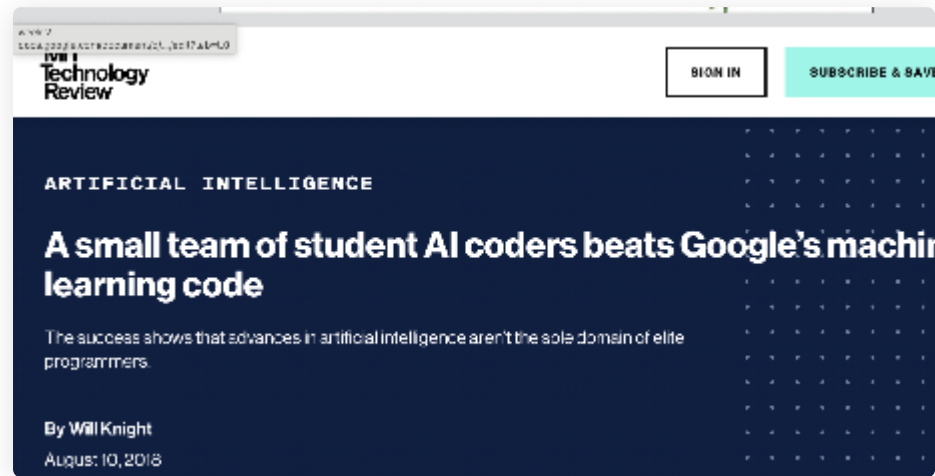
- 6nd-7th time implementing backprop by hand

Lesson: backprop sucks when memory is expensive

2018 DAWN Imagenet competition

Joined forces with a couple of talented juniors and beat Google at "who can train ImageNet to 75% the fastest, with no limit on compute used".

With a bright mobile engineer from fast.ai, Andrew Shaw



The code needed to run the learning algorithm on several machines was developed by a collaborator at the Pentagon's new Defense Innovation Unit, created recently to help the military work with AI and machine learning.

- <https://www.technologyreview.com/2018/08/10/14/team-of-ai-coders-beats-googles-code/>
- <https://github.com/cybertronai/imagenet18>
- <https://dawn.cs.stanford.edu/dawnbench>

Cited as an "unnamed collaborator working for US military"

Lesson: fast iteration >> thinking about beautiful ideas for a while

2019 Transformer-XL

GPT-2 reproduction

Scaling Transformer-XL to 128 GPUs



Yaroslav Bulatov

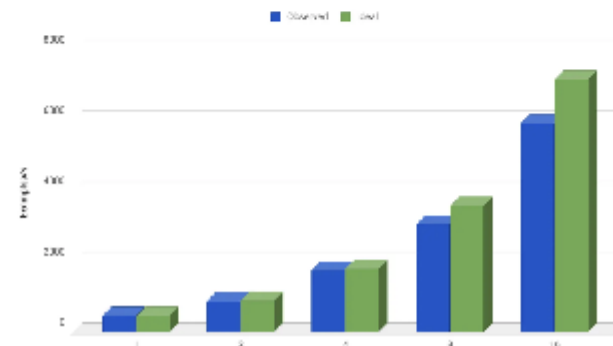


11 min read · May 6, 2018



Yaroslav Bulatov, Ben Mann, Darius Lam

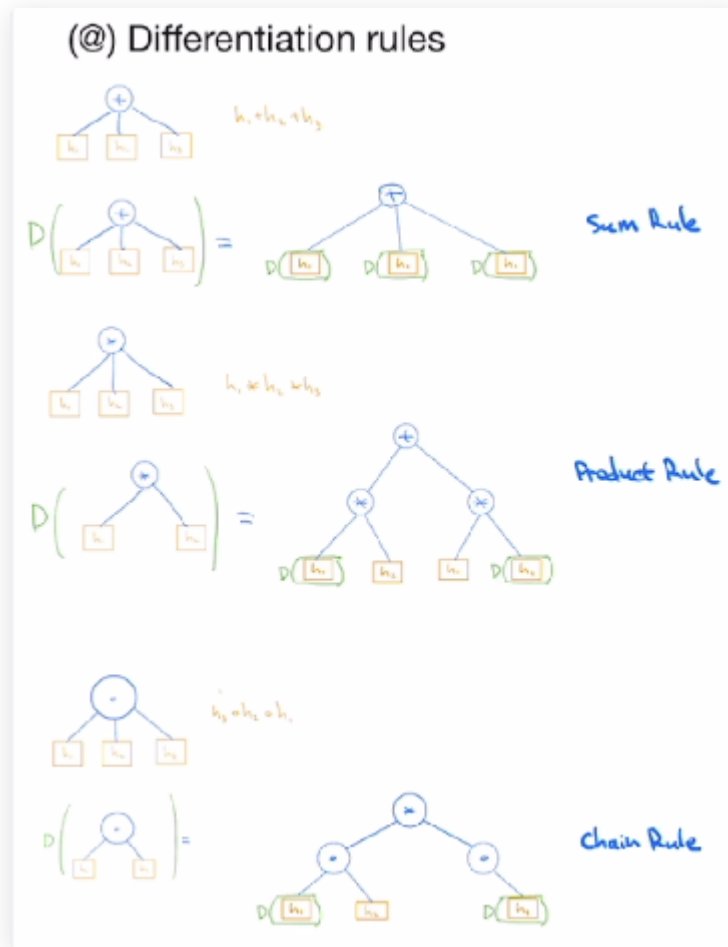
TLDRE: we made Transformer-XL train efficiently on 128 GPUs on Amazon cloud. The code is available at <https://github.com/yaroslavb/transformer-xl>



Led to first LLM code completion system in JetBrains.

2023 PyTorch Distributed team

Symbolic Differentiation engine, NCCL migration



[colab](#), [wolfram community post](#)

- 7th-10th time implementing backprop by hand

2025 Scaling Laws from Free Probability

Together.AI

Impractical Research talk

Scaling Laws with Chris De Sa, Thomas Ahle,
Chris Re. [preprint](#)



Stochastic Optimization Under Power-Law Spectra: Tight Bounds and Shuffling Analysis

Thomas Dybdahl Ahle
Normal Computing

THOMAS@NORMALCOMPUTING.COM

Yaroslav Bulatov
Together AI

YAROSLAV@TOGETHER.AI

Christopher De Sa
Cornell University

CMD353@CORNELL.EDU

Editor:

Abstract

Recent work has established that power law spectral conditions on data enable tight convergence bounds for deterministic gradient descent, resolving the conflict between classical exponential bounds and observed power law learning curves. In this work, we extend this result to the stochastic regime of high-dimensional machine learning. We provide three main contributions: (1) We generalize the power-law spectral theory to Stochastic Gradient Descent (SGD), showing that the same spectral exponents govern stochastic dynamics; (2) For the fundamental case of isotropic Gaussian data, we provide a precise analysis of data shuffling, deriving exact constants that prove Single Shuffle is strictly superior to Flip-Flop and IID sampling; (3) We conjecture and provide empirical evidence that this “shuffling gap” is a universal feature of power law spectra. Our results bridge the gap between abstract spectral theory and practical stochastic training choices, offering a unified picture of how data geometry drives optimization speed.

Keywords: Stochastic Gradient Descent, Power-Law Convergence, Spectral Analysis, Shuffling, Free Probability

Lesson: research as a whole is useful, the value of any individual paper is marginal (still fun though)

2026 Sutro Group

Sutro group: solve AI using AI

SutroYaro

Hinton Problems [repo](#)

Schmidhuber Problems [repo](#)

SutroAna

matmul



Jürgen Schmidhuber reposted this



Sakana AI

2d · Edited · 

Reproducing all of Jürgen Schmidhuber's papers (1990-2025) using an AI coding assistant. Cool project by Yaroslav Bulatov! It even... more

Schmidhuber Presents

2018-2025 — World models, fast-weight Transformers, systematic generalization

He & Schmidhuber (2018) — Recursive World Models

world-models-arc4eag

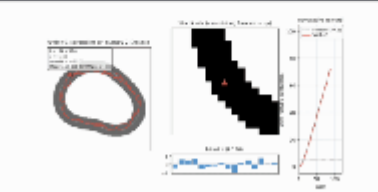
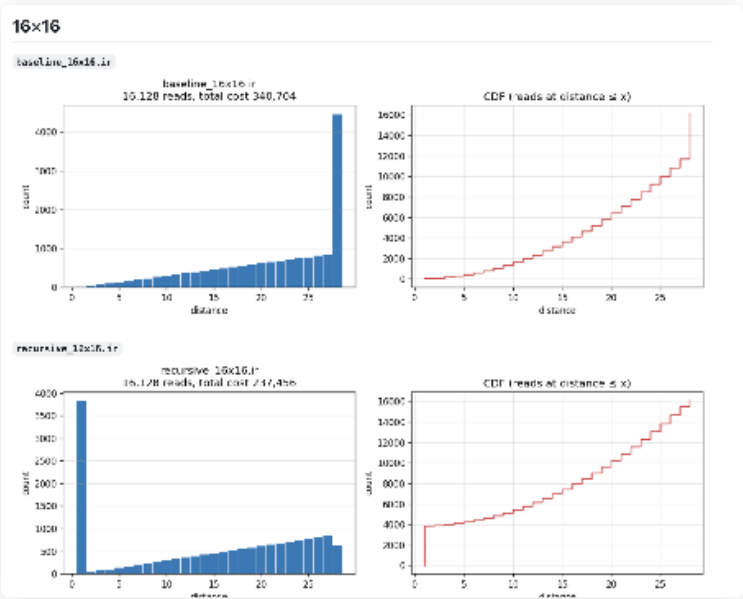


Figure 2: A sequence of images generated by the World Models arc4eag model, showing a ring world environment. The images are generated from a sequence of frames, and the model is able to generate a sequence of frames that are consistent with the previous frames. The model is able to generate a sequence of frames that are consistent with the previous frames, and the model is able to generate a sequence of frames that are consistent with the previous frames.

world-models-arc4eag



sparse parity

	bits	sparse parity
train	$\left\{ \begin{array}{l} 1 \ 0 \ 1 \ 0 \ 0 \\ 1 \ 0 \ 0 \ 0 \ 1 \end{array} \right.$	$\begin{array}{l} 1 \\ 0 \end{array}$
test 8x larger	$\left(\begin{array}{l} 1 \ 0 \ 0 \ 0 \ 1 \\ 1 \ 1 \ 1 \ 0 \ 0 \end{array} \right.$	$\begin{array}{l} ? \\ ? \end{array}$
	$\left(\begin{array}{l} 0 \ 1 \ 0 \ 1 \ 1 \\ 0 \ 0 \ 1 \ 1 \ 0 \end{array} \right.$	$\begin{array}{l} ? \\ ? \end{array}$
	$\left(\begin{array}{l} 1 \ 0 \ 1 \ 1 \ 0 \\ 0 \ 1 \ 1 \ 0 \ 1 \end{array} \right.$	$\begin{array}{l} ? \\ ? \end{array}$

wiktext

Lesson: agents can one-shot swathes of research now

Examples of Path Dependencies

What is path dependency?

1. make a **good** design decision
2. the environment changes
3. decision is now **bad**

Puzzle







Nuclear Reactors

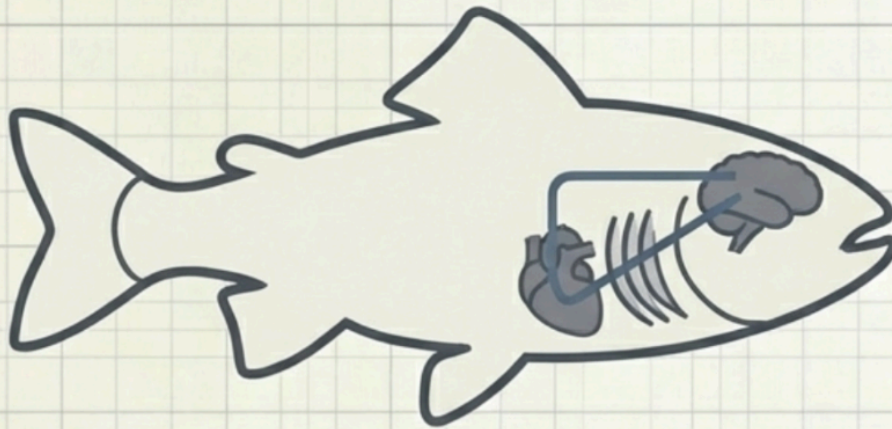
- **Nuclear Power: The "Submarine" Trap**

- **The Lock-in:** Light Water Reactors (LWR) are the standard for nuclear power, but they are not the safest or most efficient design. They became the standard because **Admiral Rickover** needed a compact reactor for *submarines*.
- **The Failure:** The civilian industry just copied the submarine supply chain. We spent 50 years optimizing a "submarine engine" for cities, ignoring better designs (like Molten Salt Reactors) because the supply chain already existed.

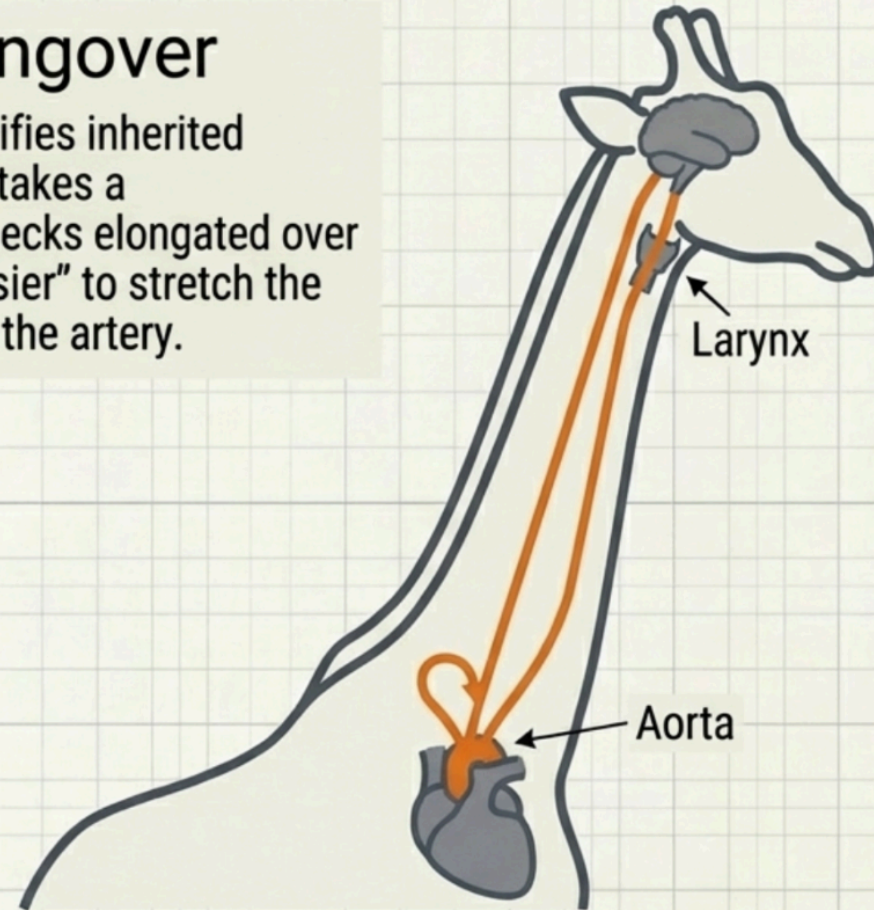
Laryngeal nerves

Biology: The Evolutionary Hangover

Natural selection cannot design from scratch; it modifies inherited structures. The recurrent laryngeal nerve in giraffes takes a preposterous 15-foot detour because, as vertebrate necks elongated over millions of years, incremental mutations made it “easier” to stretch the nerve than to structurally sever and reroute it across the artery.

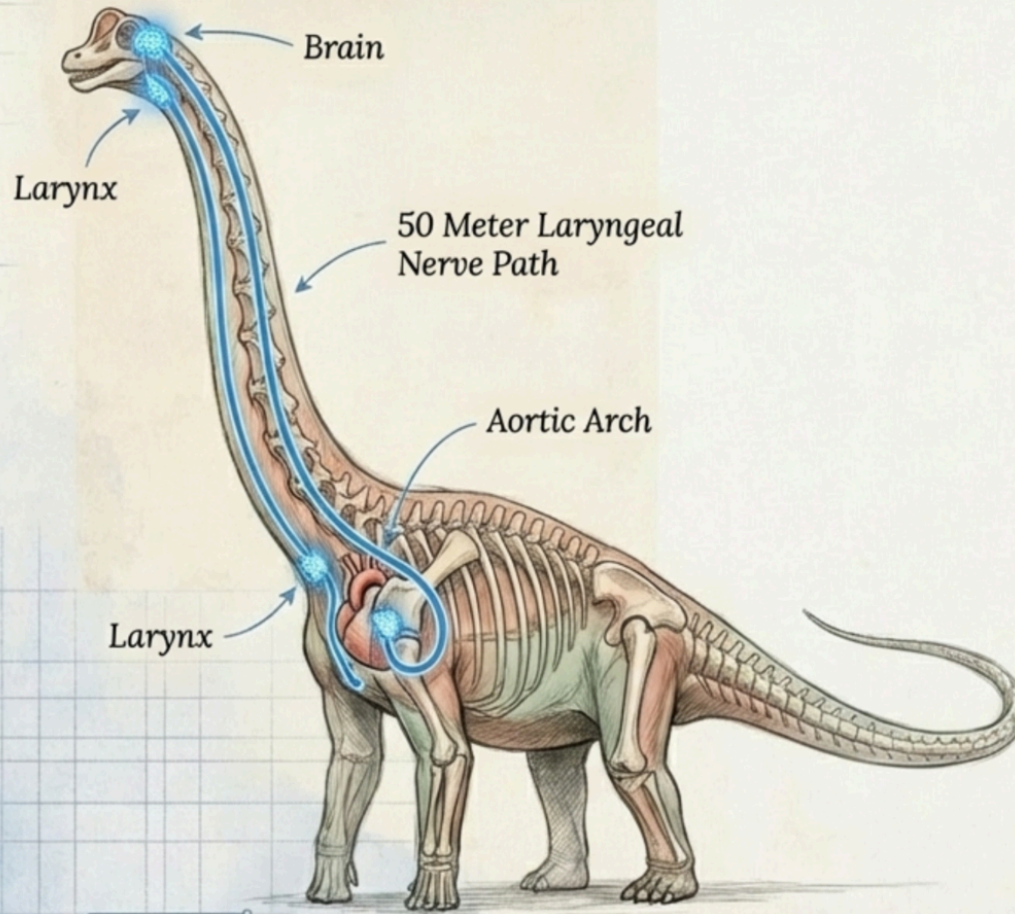


Fish: Direct neural path.



Giraffe: 15-foot path-dependent detour.

The Ultimate Bottleneck: The Sauropod Nerve



The Flaw

The longest single cell in the history of life.

The Origin

In early fish, this nerve ran a few inches straight from the brain to the gills. As necks lengthened over millions of years, the nerve was already trapped under the heart's aortic arch.

The Result

50 meters of wasted distance to travel a few inches of actual space.
Inefficiency by design.

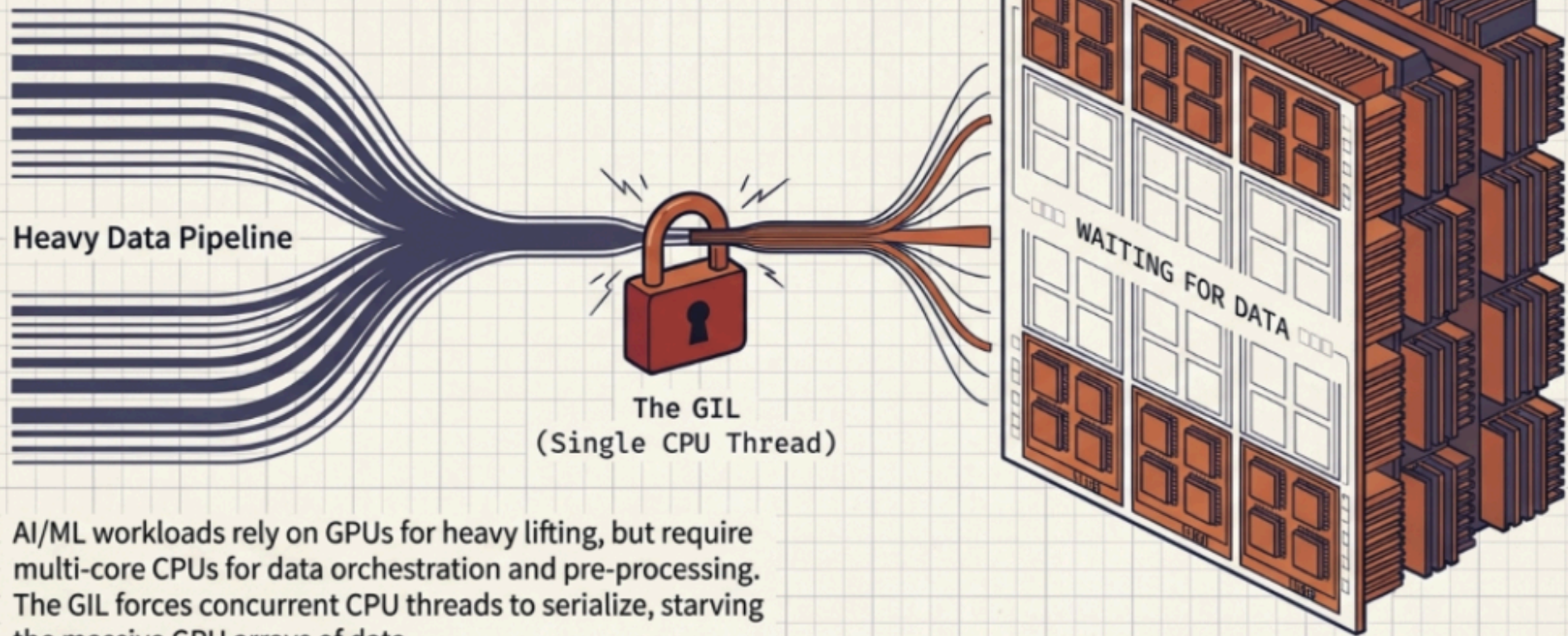
Old embryonic morphogenesis program

New anatomic dimensions

Embryonic morphogenesis program **now obsolete** (but hard to change)

Python Global Interpreter Lock

The Multi-Core Crisis in Modern AI



AI/ML workloads rely on GPUs for heavy lifting, but require multi-core CPUs for data orchestration and pre-processing. The GIL forces concurrent CPU threads to serialize, starving the massive GPU arrays of data.

2005 multi-core machines

2030 no-GIL by default (if we are lucky)

Path dependencies in Deep Learning

What is path dependency?

1. make a good design decision
 2. the environment changes
 3. decision is now bad
- Design decisions made in the old era
 - generic components: losses, optimizers, activations
 - sequential dependencies: layers, gradient descent.
 - low compute-to-commute ratio: backprop
 - The environment changed
 - A. shifting hardware
 - B. narrowing of application

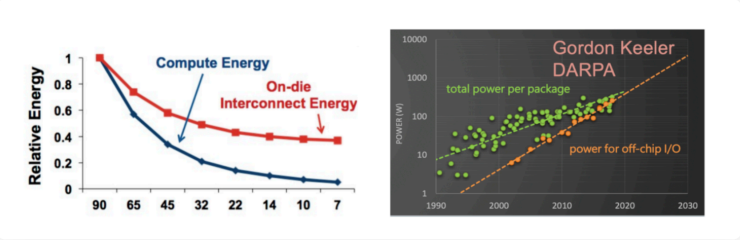
Environment change

Hardware drift

The Memory wall

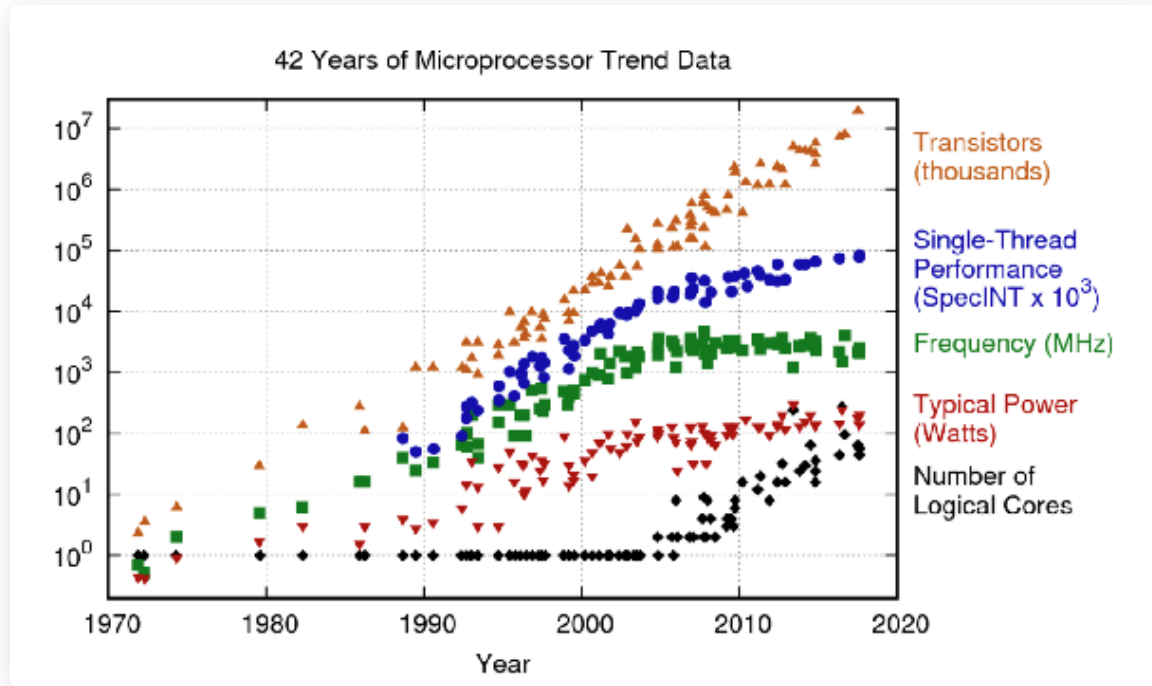
Period	Arithmetic improvement	Memory improvement	Gap growth
Pre-mid-1980s CPUs	~22%/yr SPEC performance	Wulf/McKee DRAM-speed assumption: ~7%/yr	~14%/yr, gap doubles ~5.3 yrs
1986–2003 CPUs	~52%/yr, doubling ~2 yrs	~7%/yr DRAM speed	~42%/yr, gap doubles ~2.0 yrs

Component	Energy	What it represents
On-chip communication	~100 fJ/bit-mm	Moving one bit one millimeter across on-chip wires
Small SRAM access	~50 fJ/bit	Reading/writing one bit from a small (local) SRAM macro
Arithmetic (add)	~1 fJ/bit	One bit of addition logic toggling



Arithmetic is now essentially free

The end of Dennard scaling



Compilers are trying to make things work, yet abstractions leak. Some algorithms are impossible to parallelize (hence minibatching)

[illegible]

Unreliable components

<https://arxiv.org/pdf/2407.21783>

The Llama 3 Herd of Models

During a 54-day snapshot period of pre-training, we experienced a total of 466 job interruptions. Of these, 47 were planned interruptions due to automated maintenance operations such as firmware upgrades or operator-initiated operations like configuration or dataset updates. The remaining 419 were unexpected interruptions, which are classified in Table 5. Approximately 78% of the unexpected interruptions are attributed to confirmed hardware issues, such as GPU or host component failures, or suspected hardware-related issues like silent data corruption and unplanned individual host maintenance events. GPU issues are the largest category, accounting for 58.7% of all unexpected issues. Despite the large number of failures, significant manual intervention was required only three times during this period, with the rest of issues handled by automation.

Applications drift

Original Deep learning didn't have an application, so the aims were broad.

Rosenblatt, (eventual) space exploration

1958, The New York Times

NEW NAVY DEVICE LEARNS BY DOING

Psychologist Shows Embryo
of Computer Designed to
Read and Grow Wiser

WASHINGTON, July 7 (UPI)
—The Navy revealed the embryo of an electronic computer today that it expects will be able to walk, talk, see, write, reproduce itself and be conscious of its existence.

The embryo—the Weather Bureau's \$2,000,000 "704" computer—learned to differentiate between right and left after fifty attempts in the Navy's demonstration for newsmen.

The service said it would use this principle to build the first of its Perceptron thinking machines that will be able to read and write. It is expected to be finished in about a year at a cost of \$100,000.

Dr. Frank Rosenblatt, designer of the Perceptron, conducted the demonstration. He said the machine would be the first device to think as the human brain. As do human beings, Perceptron will make mistakes at first, but will grow wiser as it gains experience, he said.

Dr. Rosenblatt, a research psychologist at the Cornell Aeronautical Laboratory, Buffalo, said Perceptrons might be fired to the planets as mechanical space explorers.

Andrew Ng, helicopter control

Autonomous inverted helicopter flight via reinforcement learning

Andrew Y. Ng¹, Adam Coates¹, Mark Diel², Varun Ganapathi¹, Jamie Schulte¹, Ben Tse², Eric Berger¹, and Eric Liang¹

¹ Computer Science Department, Stanford University, Stanford, CA 94305

² Whirled Air Helicopters, Menlo Park, CA 94025

Abstract. Helicopters have highly stochastic, nonlinear, dynamics, and autonomous helicopter flight is widely regarded to be a challenging control problem. As helicopters are highly unstable at low speeds, it is particularly difficult to design controllers for low speed aerobatic maneuvers. In this paper, we describe a successful application of reinforcement learning to designing a controller for sustained inverted flight on an autonomous helicopter. Using data collected from the helicopter in flight, we began by learning a stochastic, nonlinear model of the helicopter's dynamics. Then, a reinforcement learning algorithm was applied to automatically learn a controller for autonomous inverted hovering. Finally, the resulting controller was successfully tested on our autonomous helicopter platform.

Lack of deep learning "product market fit" means we focused on components reusable across domains

- blackbox optimizers

- generic loss functions
- reusable components (ReLU)

Design decisions

What design decisions were made in the old environment?

Gradient descent

Black box optimizers (designed for shifting demands)

Hit a dead-end.

Descending a crowded valley

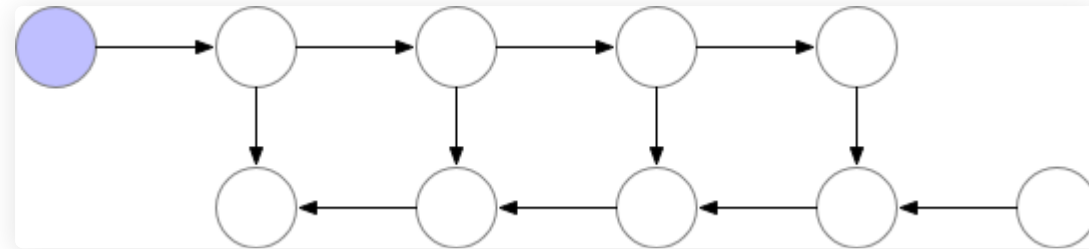
A. List of optimizers and schedules considered

Table 2: List of optimizers considered for our benchmark. This is only a subset of all existing methods for deep learning.

Name	Ref.	Name	Ref.
AdaboGrad	(Geary et al., 2011)	HyperAdapt	(Wang et al., 2018b)
AC3Clip	(Zhang et al., 2022)	JB-BFGS-KMPG(SL)	(Goldfarb et al., 2020b)
AdaBelter	(Xie et al., 2019)	KF-QN-CNN	(Reis & Goldfarb, 2021)
AdaBelter	(Demirakanda et al., 2017)	KFAC	(Martens & Grosse, 2015)
AdaBelter/AdaBelter-S5	(Aichison, 2022)	OPT-BKTR	(Bauer et al., 2017)
AdaBelter	(Zhong et al., 2022)	1-Lambda-Momentum	(Redlich & Moritz, 2018)
AdaBlock	(Yan et al., 2018)	LAMD	(You et al., 2020)
AdaBound	(Luo et al., 2018)	LaDap	(Zhu et al., 2020)
AdaCovp	(Chen et al., 2014)	LARS	(You et al., 2017)
AdaDelta	(Zeiler, 2012)	LIGPT	(Alamir et al., 2021)
AdaHessian	(Shazeer & Sutskever, 2014)	LookAhead	(Zhang et al., 2019)
AdaTix	(Bao et al., 2019)	M-SVAG	(Bailes & Herzig, 2018)
AdaPam	(Chen et al., 2019a)	MADGRAD	(Delella & Jelassi, 2021)
AdaPTT	(Delella & Jelassi, 2021)	NAS	(J. Schwab et al., 2020)
Adagrad	(Duchi et al., 2011)	NFRA	(Chen et al., 2020)
ADAMHESIAN	(Yao et al., 2022)	MFAdam	(Malliet & Wolf, 2020)
Afri	(Xie et al., 2020)	MVRC-1/MVRC-2	(Chen & Zhou, 2020)
AdaLoss	(Tosoni et al., 2019)	Nadam	(Baldi, 2016)
Adam	(Kingma & Ba, 2015)	NAMES/NAMSG	(Chen et al., 2019b)
Adam-I	(Jin et al., 2020c)	ND-Adam	(Zhong et al., 2017a)
AdamAF	(Bao et al., 2019)	Nico	(Jin et al., 2021a)
AdaMax	(Kingma & Ba, 2015)	Noisew	(Sennov, 1983)
AdamSE	(Jin et al., 2020c)	Noisy-Adam/Noisy-K-FAC	(Zhong et al., 2018)
AdamNC	(Reddi et al., 2018)	NoAdam	(Huang et al., 2019)
AdaMod	(Ding et al., 2019)	Novograd	(Ginsburg et al., 2019)
AdamPSDHP	(Hou et al., 2021)	NLSGD	(Zhou et al., 2021a)
AdamT	(Zhou et al., 2022)	Padam	(Chen et al., 2022a)
AdamW	(Loshchilov & Hutter, 2019)	PAGE	(Jin et al., 2021b)
AdamX	(Tran & Phung, 2019)	PAI	(Marschler & Zell, 2020)
AdAS	(Tijssen, 2022)	PolyAdam	(Grosven et al., 2019)
Adas	(Hossein, S. & Hattami, 2022)	Polyak	(Polyak, 1991)
AdaScale	(Johnson et al., 2022)	PowerGDM/PowerGDM	(Vogel et al., 2019)
AdaSGD	(Wang & Wiecek, 2022)	Probabilistic Polyak	(de Rooi et al., 2021)
AdaShft	(Zhou et al., 2019)	ProBL3	(Mishchenko & Herzig, 2017)
AdaSpit	(Shi et al., 2019)	Pyram	(Xu, 2020)
Adaptive	(Shi et al., 2019)	QHAdapt/QHM	(Ota & Yamazaki, 2019)
AdaX/AdaX-W	(Jin et al., 2020b)	RAdam	(Liu et al., 2020a)
AFGD	(Jin & Tian, 2020)	Range1	(Wright, 2021b)
ALIC3	(Bernado et al., 2020)	RangeAdam	(Grosven, 2020)
AMSR-sand	(Jin et al., 2019)	RMSProp	(Tieleman & Hinton, 2012)
AMSGrad	(Reddi et al., 2018)	RMSterev	(Chen et al., 2019)
AngularGrad	(Roy et al., 2021)	S-GD	(Sang et al., 2020)
AmuJALS	(Varvada et al., 2019)	SAdam	(Wang et al., 2021b)
ARMS	(Cohen et al., 2019)	SAdam/NAMSEnd	(Ding et al., 2019)
ASAM	(Kwon et al., 2021)	SALR	(Yue et al., 2020)
Asot-RS	(Jin et al., 2021)	SAM	(Peters et al., 2021)
AsaGrad	(Souto et al., 2019)	SG-Adagrad/SG-RMSProp	(Mukkamala & Hsu, 2017)
BAdapt	(Sato et al., 2018)	SGDwp	(Sato et al., 2017)
BGAdam	(Bai & Zhang, 2019)	SGD	(Robbins & Monro, 1951)
BPGraf	(Zhang et al., 2017c)	SGD-LB	(Tan et al., 2018)
BRMSProp	(Aichison, 2022)	SGD-G2	(Agadi & Turinici, 2020)
BSGD	(Shi et al., 2022)	SGDRM	(Rauwous, Kolyer et al., 2021)
C-ADAM	(Hannun et al., 2022)	SGD-H14	(Jin & Cukras, 2021)
CMA	(Chen et al., 2021)	SHAM	(Jin & Liu, 2020)
Coal Momentum	(Barysenko & Pyshkin, 2022)	SGDR	(Lechelt & Hutter, 2017)
CProp	(Prasadhar & K. S. Reddy, 2019)	SRAdapt	(Phung et al., 2020)
CurvBall	(Herrnstein et al., 2018)	SharpNet	(Anil et al., 2022; Gupta et al., 2018)
DAdam	(Nasari et al., 2019)	SignAdam++	(Wang et al., 2016a)
DregMemory	(Wright, 2020a)	SignSGD	(Herrnstein et al., 2018)
DGN-Opt	(Liu et al., 2021a)	SKQN5-QN	(Yang et al., 2020)
DREGRad	(Loshchilov et al., 2022)	SMP	(Arai et al., 2019)
DAdapt	(Yuan & Gao, 2022)	SMG	(Tan et al., 2020)
KKFAC	(Georgio et al., 2014)	NNCM	(Lian et al., 2020)
Eve	(Hayasaka et al., 2014)	SoftAdapt	(Herrnstein et al., 2019)
ExpSignad	(Tobin & Asano, 2022)	SRSGD	(Wang et al., 2020a)
FastAdaptiveNet	(Zhou et al., 2021a)	Step-Limited SGD	(Laurin et al., 2021)
FRSGD	(Wong & Ye, 2020)	SWATS	(Kocher & Socher, 2017)
O-Adaptad	(Chakrabarti & Gupta, 2021)	SVN123	(Cao et al., 2019c)
GAAdAM	(Zhang & Usuya, 2018)	TAdam	(Herrnstein et al., 2020)
Hadam	(Hannun et al., 2022)	TKFAC	(Tan et al., 2020)
QUALS	(Chen et al., 2021)	VAdam	(Khan et al., 2018)
OGF-S3	(Kotik & Wilco, 2019)	VR-SGD	(Zhong et al., 2020)
Onk-Arg	(Pukayipat & Puthanur, 2022)	vSGD/MSGD/MSGD-L	(Schulz et al., 2018)
CRATPS	(DellaGrossa et al., 2021)	vSGD-M	(Schulz & Lofman, 2013)
OmniCon	(Kohsuri et al., 2022)	WNGrad	(Wu et al., 2018)
OmniSty	(Eshwari & Zadeh, 2021)	YellowFin	(Zhong & Milliken, 2019)
Radam	(Liang et al., 2019)	Yogi	(Gutier et al., 2018)

Backprop

Targetting low compute/commute ratio



Synchronous training

Good for reliable set of components

Story of async/sync switch at Google

- Original models trained async. Then switched to sync shortly after GPU training.

Anecdotal it happened as follows:

- Shubho Sengupta got it working in Baidu
- Andrew Ng told Jeff Dean
- Jeff Dean told Google Brain folks to try it.

Ultimately reasons for the switch were as follows:

- 1. minor improvement in accuracy on the ImageNet
- 2. sync runs were deterministic, making debugging

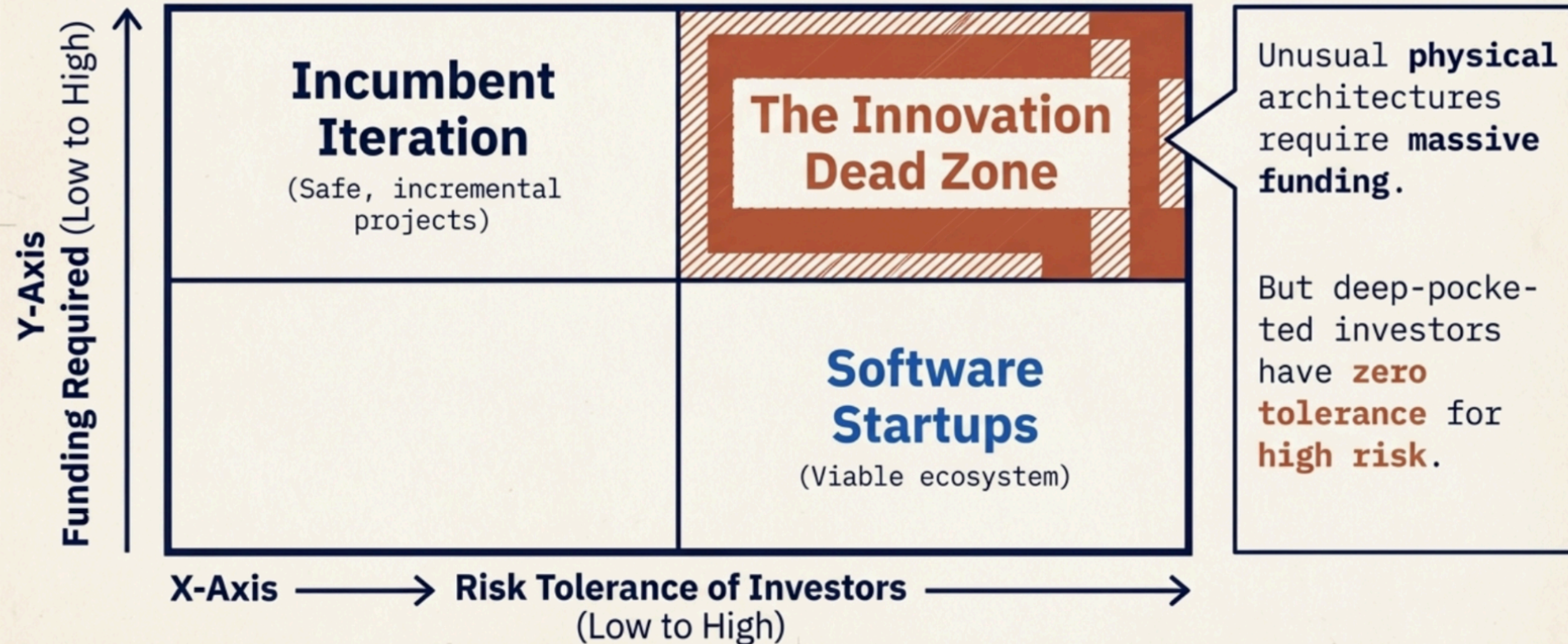
Things which hard to change

Expensive things.

- Large ambition requires large capital
- Large capital is conservative.
- Hence “kill zone”:

technology that is promising but not bankable.

Capital markets fundamentally incompatible with hardware risk profiles



2018 Cade Metz article



45 AI hardware startups.

300->450 wafer upgrade story

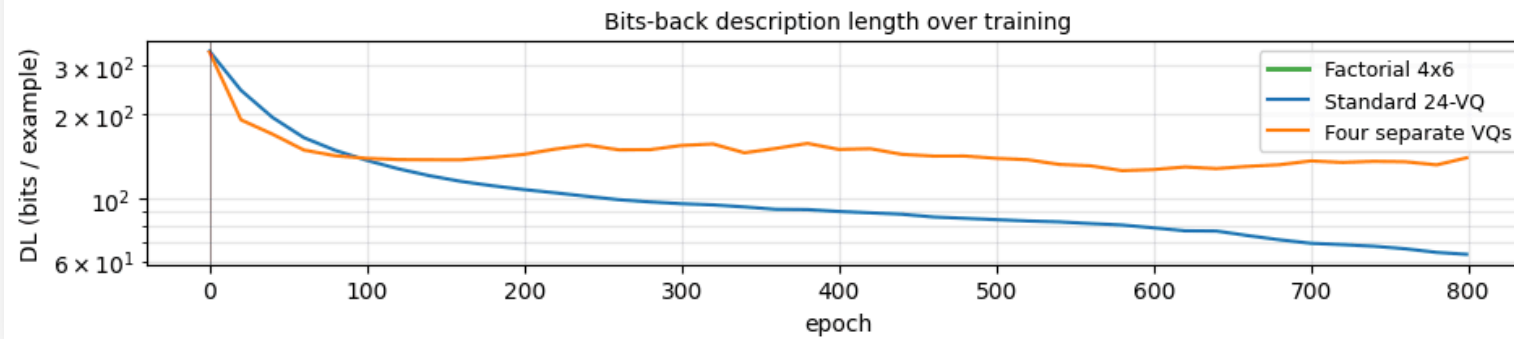
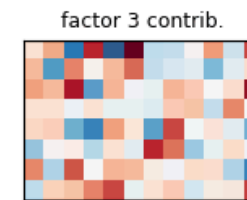
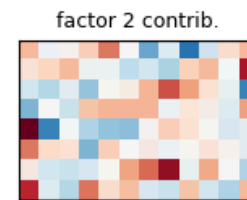
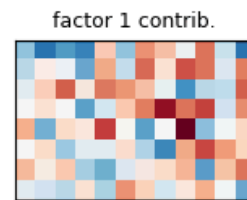
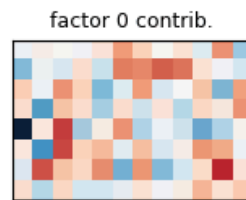
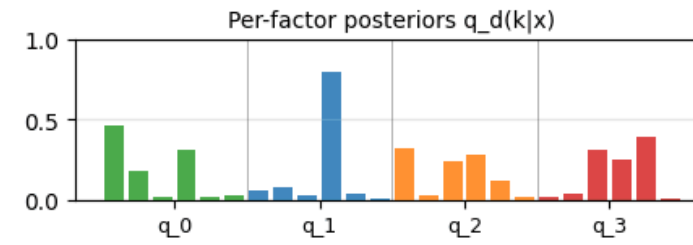
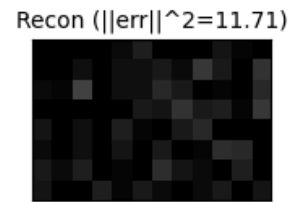
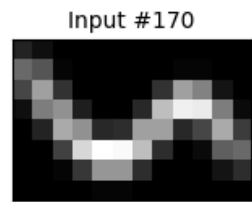
Things which easy to change

Cheap things.

This year: Learning algorithm development.

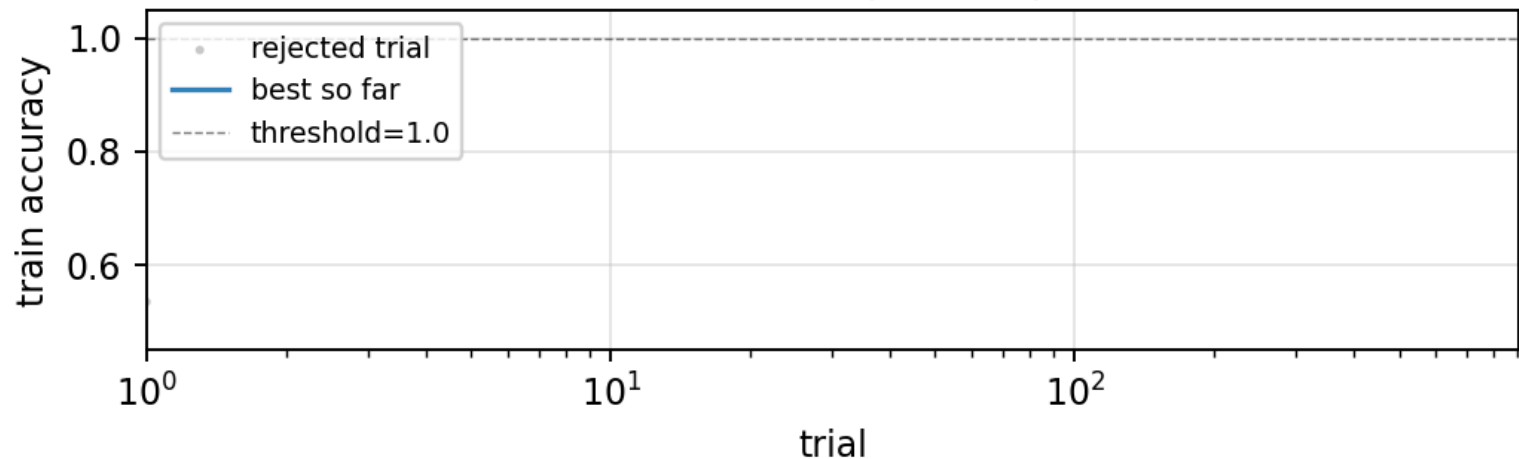
Implement 55 Hinton papers: 1.2B tokens ([visual_tour](#))

Spline images, factorial VQ — epoch 0

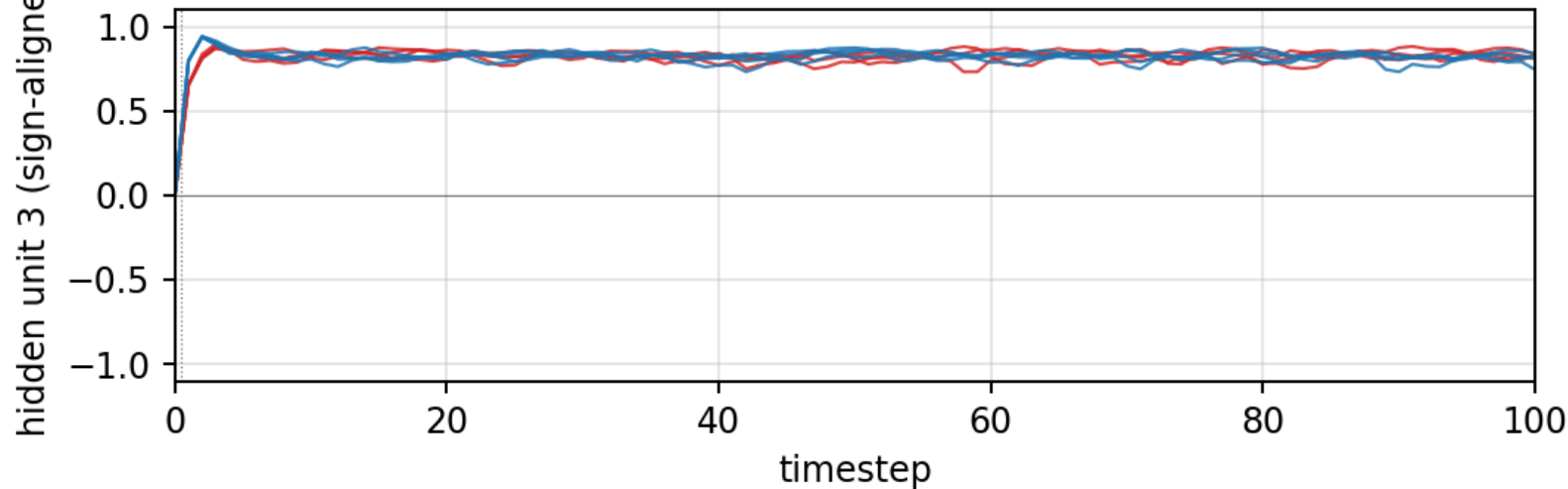


Implement 58 Schmidhuber papers: 1.5B tokens [visual_tour](#)

random-weight-guessing on Bengio-94 latch | trial 1 | best train 0.535 (found at



activation behavior of best-so-far net (red: class +1, blue: class -1) — train 0.54 te



The future

Redeveloping of algorithms (**easy to change**)

for a better fit

to existing hardware (**hard to change**)

Potential Examples

Which things become possible with AI agents?

Fewer abstraction splits

Why do we have abstractions?

n^2 communication bottleneck.

If a task requires more than x people, better to split.

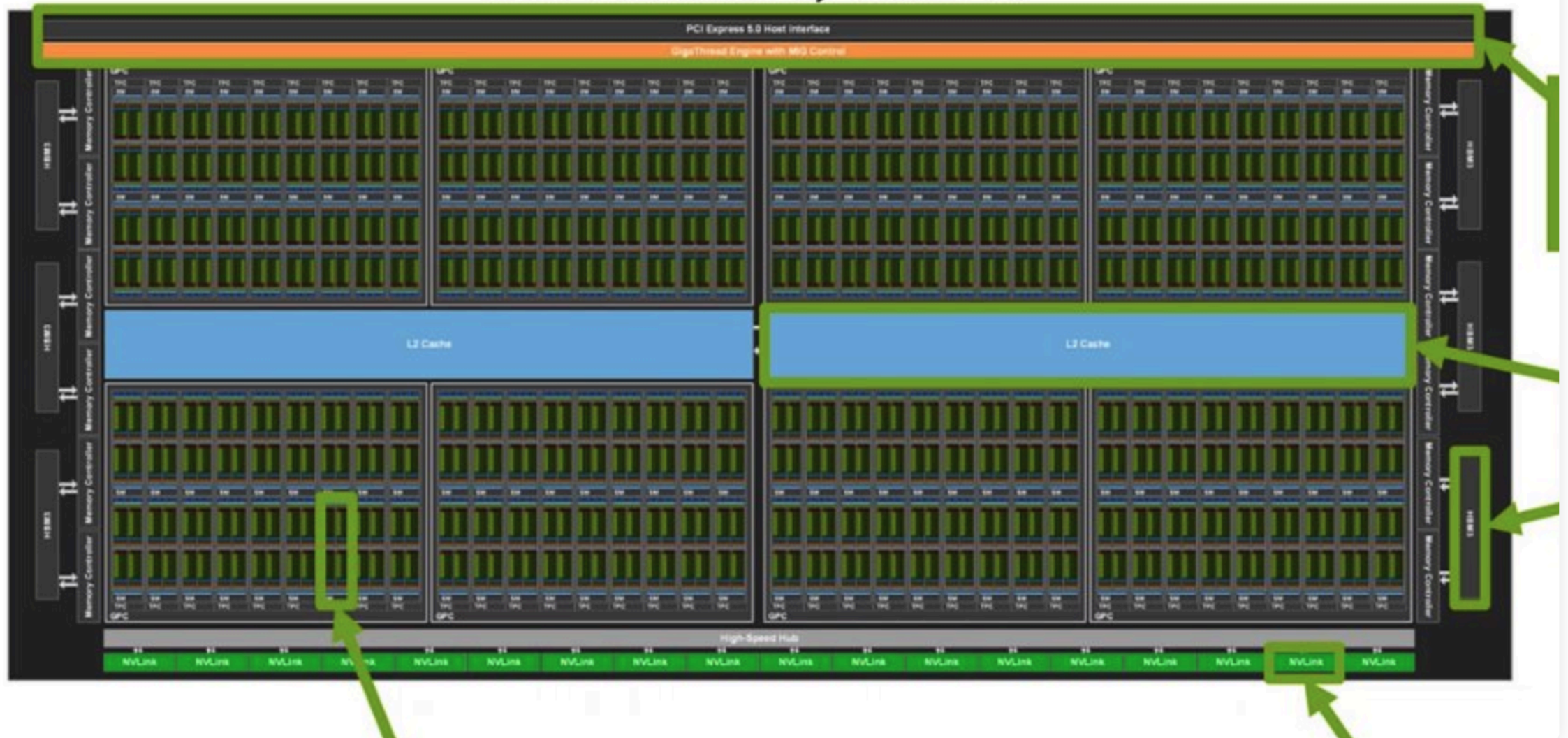
Capable people = Less people needed = less splits.

Less splits = more freedom to improve things!

Mega-kernels

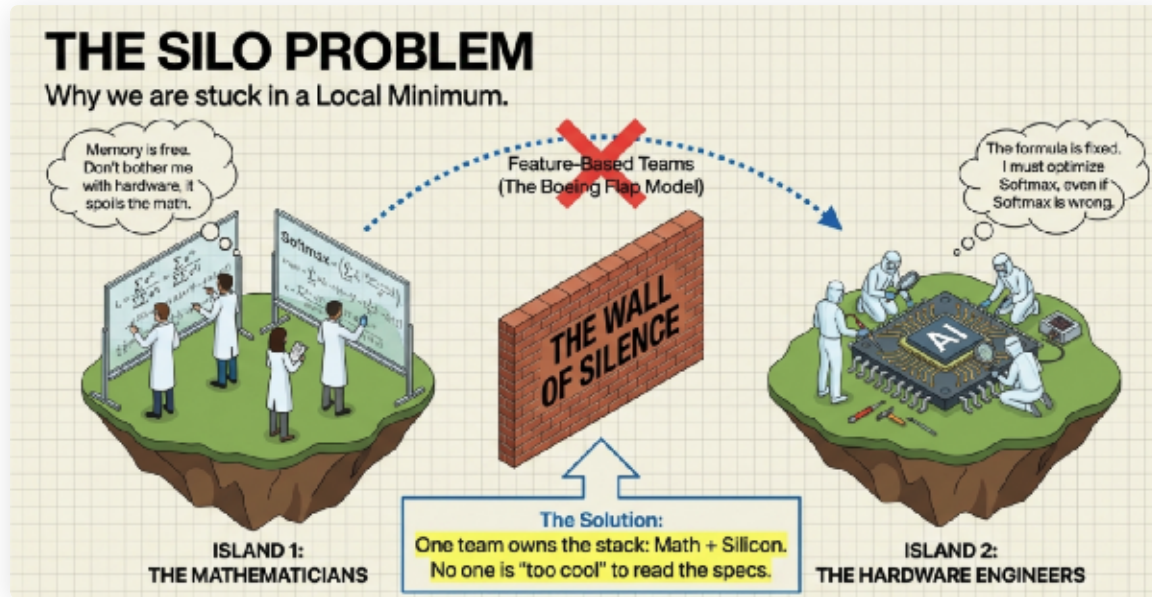
framework/kernel wall split

80B Transistors, TSMC 4N



Hardware-specific algorithms

math/compiler wall split



Old world = slow feedback.

Mini-batching motivated by the memory wall.

(most researchers don't know that)

2003 - "The general inefficiency of batch training for gradient descent learning"

Abstract

Gradient descent training of *neural networks* can be done in either a batch or on-line manner. A widely held myth in the neural network community is that batch training is as fast or faster and/or more 'correct' than on-line training because it supposedly uses a better approximation of the true gradient for its weight updates. This paper explains why batch training is almost always slower than on-line training – often orders of magnitude slower – especially on large training sets. The main reason is due to the ability of on-line training to follow curves in the error surface throughout each epoch, which allows it to safely use a larger learning rate and thus converge with less iterations through the training data. Empirical results on a large (20,000-instance) speech recognition task and on 26 other learning tasks demonstrate that convergence can be reached significantly faster using on-line training than batch training, with no apparent difference in accuracy.

2006 -- "A Fast Learning Algorithm for Deep Belief Nets"

<https://www.cs.toronto.edu/~fritz/absps/ncfast.pdf>

work was 1.25% errors on the official test set. The network shown in Figure 1 was trained on 44,000 of the training images that were divided into 440 balanced mini-batches, each containing 10 examples of each digit class.⁶ The weights were updated after each mini-batch.

Explosion of DSLs this year

Timeline view

2021–2023: Triton became the productivity default for custom ML kernels; CUDA/CUTLASS remained the expert-performance default.

2024: Hopper/H100 exposed the limits of first-generation Triton for peak attention. FlashAttention-3 used CUTLASS/CuTe primitives, WGMMA, TMA, warp specialization, and FP8. GPU Mode's public materials around this period include CUTLASS, FlashAttention-3, FlashInfer, Mosaic GPU, and CuTe. [arXiv +2](#)

2025: Tile DSLs proliferated. Tilus, TileLang, ThunderKittens, Helion, Gluon/TLX, Cypress, Tawa, DeepGEMM, and Mirage/MPK all fit the same pattern: more specialization, more explicit scheduling, and/or more compiler-managed orchestration. [GitHub +4](#)

2026: Blackwell/B200 made the trend unavoidable. FlashAttention-4 is CuTe DSL; CUDA Tile/TileIR becomes a visible NVIDIA compiler substrate; DeepSeek-V4 support uses TileLang kernels; ThunderKittens 2.0 adds Blackwell/MXFP8/NVFP4. [arXiv +3](#)

Novel learning algorithms



Backprop compute-to-commute ratio suits for pre-memory-wall era.
What is natural for the post-memory-wall era?

Architecture/optimizer co-design

Scientific computing literature customizes equation solvers to the structure of the problem.

- multigrid methods
- fast multipole
- etc.

Deep Learning uses the same optimizer for convolutional networks as for Transformers.

- going from blackbox to architecture specialized

ex: Adam is blackbox, Muon is not blackbox

Approach

- Repeat old approach
- But faster

(we spent 70 years trying semi-random things on toy problems)

Path to attention

- Deep Sets (Zaheer et al. 2017)
- Deep Sets with Attention aka Multi-Instance Learning (Ilse, Tomczak, Welling, '18)
- Bag of words (Salton & McGill, 1986) Word2Vec (Mikolov et al., 2013)
- Attention weighting for documents (Wang et al, '16)
- Hierarchical attention weighting (Yang et al. '17)
- Question Answering with Pooling and Iteration (Sukhbaatar et al., '15)
- Seq2Seq with attention (Bahdanau, Cho, Bengio '14) (Pham, Luong, Manning '15)
- Pointer networks for finding convex hull (Vinyals et al., '15)
- Transformer with multi-head attention (Vaswani et al., '17)

(from Alex Smola attention [talk](#))

Sutro Group

agentic research:

[SutroYaro](#)

[SutroAna](#)

hill-climbing:

theory track (can we make learning memory friendly?):

[matmul](#)

[sparse-parity](#)

applied track (measure actual Joules)

[wikitext](#)

BarCode

